

Project Specification: Accessible Personalized Multisensory Reading Assistant

Project Goal

Build the **frontend-only MVP/prototype** of an accessible dyslexia-support web application.

The application is a personalized reading environment rather than a diagnostic or medical application.

The core idea is:

Upload/read content → personalize the reading experience → optionally translate → read aloud → focus/highlight → save personal preferences.

The frontend must be fully interactive and usable with realistic mock data.

IMPORTANT ARCHITECTURE REQUIREMENT

Do **NOT** implement the real backend, database, Sarvam API, authentication backend, MCP server, or external AI APIs yet.

Instead:

1. Build the complete frontend experience.
2. Implement frontend-only features for real.
3. Implement backend-dependent features using mock services.
4. Create a clean service/API abstraction layer.
5. Make it possible to replace mock services with a future FastAPI/Sarvam backend without rewriting the UI.

The eventual architecture will be:

```
Next.js Frontend
  ↓
Service/API abstraction
  ↓
Future FastAPI Backend
  ↓
Sarvam AI / Database / AI services
```

Do not tightly couple React components to mock implementations or future Sarvam APIs.

1. TECH STACK

Use:

- Next.js
- App Router
- React
- TypeScript
- Tailwind CSS
- React Context or Zustand for application state
- Web Speech API for prototype TTS
- pdfjs-dist or react-pdf for PDF handling
- localStorage for prototype persistence

Structure the project so a future FastAPI backend can replace mock services.

2. GLOBAL DESIGN SYSTEM & ACCESSIBILITY BASELINE

IMPORTANT

The following colour, typography, spacing, sizing, and contrast requirements have been deliberately researched for this project.

Do not remove, weaken, or replace these requirements with generic design defaults.

Background Palette

Strictly avoid pure white (#FFFFFF).

Use soft, anti-glare off-white/cream tones such as:

• **FBF9F5**

• **F4F1EA**

Color Scheme

Use muted, soothing, low-saturation pastel/earth tones such as:

- soft sage

- slate blue
- warm gray

Avoid jarring, saturated, or overly bold colors.

Contrast Compliance

Ensure all text-to-background combinations meet WCAG accessibility requirements.

Target:

- minimum 4.5:1 for standard text
- minimum 3:1 for large text

Default Typography

Use Arial for initial system screens.

Reading Typography

The reading system must preserve the researched typography settings and allow user personalization.

Do not impose one "dyslexia font" as universally best.

3. SPECIAL PERSONALIZED READING COLOR SYSTEM

Color personalization is a **core product feature**.

Do not assume one background color is universally best for users with dyslexia.

Provide multiple reading themes that users can test and select according to comfort.

Example themes:

Warm Cream

Background:

FBF9F5

Soft Yellow

Use a very pale, low-saturation yellow.

Soft Green

Use a very pale, low-saturation green.

Soft Blue

Use a very pale, low-saturation blue.

Also support:

Custom Theme

Allow users to customize:

- background color
- text color
- highlight color

The selected reading theme must immediately affect the reading area.

The application UI itself can maintain a consistent brand palette, while the **document reading area is independently themeable**.

Do not use neon or highly saturated highlighting.

4. APPLICATION STRUCTURE

Create these primary routes:

```
/
/login
/onboarding/assessment
/profile
/library
/reader
/settings
```

If appropriate, use dynamic routes such as:

```
/reader/[documentId]
```

5. LANDING PAGE (/)

Create a polished landing page.

Main message:

"Read in the way that works for you."

Supporting message:

"Personalize text, colors, spacing, language, focus and audio to create a reading experience that works for you."

Primary CTA:

Start Reading

Secondary CTA:

How It Works

Feature highlights:

- Personalized Reading
- Multilingual Reading
- Read Aloud
- Focus Mode
- Reading Calibration
- Accessible Colors
- Adaptive Formatting

The page should be calm and accessible rather than looking like a generic AI SaaS website.

6. LOGIN PAGE (/login)

Create a minimalist login interface.

For this frontend prototype:

- Do not implement real authentication.
- Use a mock login.
- Simulate successful login.
- Store a mock user ID locally.

The UI should be designed so Firebase/Auth/Supabase can later replace the mock authentication.

7. OPTIONAL PERSONALIZED DYSLEXIA PROFILE

This is an **optional feature**.

Do not make uploading an assessment/prescription mandatory.

The user should be able to use the application without uploading any assessment.

Provide an optional flow:

```
User
↓
Optional assessment document
↓
Mock document processing
↓
Potential profile information
↓
User confirms preferences
```

IMPORTANT:

Do not diagnose dyslexia.

Do not make medical claims.

Treat assessment/prescription information as user-provided information.

The frontend should clearly explain that the document is optional.

8. READING CALIBRATION

Create a complete interactive reading calibration system.

Purpose:

Find a practical starting reading configuration based on what the user finds easiest to read.

Do not claim that one configuration is medically optimal.

Calibration Flow

```
graph TD; A[Start Calibration] --> B[Sample comparison]; B --> C[User chooses easiest version]; C --> D[Multiple rounds]; D --> E[Generate preferred settings]; E --> F[Show profile summary]; F --> G[Apply preferences];
```

Each sample should use the same text but different combinations of:

- font
- font size
- boldness
- letter spacing
- word spacing
- line spacing
- background color
- text color
- highlighting
- text width

Use the project's researched typography/spacing requirements.

Create multiple calibration rounds.

Example:

Round 1

Choose preferred typography.

Round 2

Choose preferred spacing.

Round 3

Choose preferred color theme.

Round 4

Choose preferred highlighting.

Round 5

Show a final comparison using the combined settings.

The user selects:

"This is easiest to read"

At the end show:

Your Reading Profile

Font: ...

Font Size: ...

Letter Spacing: ...

Word Spacing: ...

Line Spacing: ...

Background: ...

Text Color: ...

Highlighting: ...

Text Width: ...

Allow:

Apply Profile

and:

Redo Calibration

For the prototype, store calibration results in localStorage.

Create a service abstraction so later this becomes:

```
POST /api/calibration/result
```

9. USER READING PROFILE

Create a persistent frontend reading profile.

Store:

```
font
fontSize
boldness
letterSpacing
wordSpacing
lineSpacing
paragraphSpacing
backgroundColor
textColor
highlightColor
textWidth
alignment
highlightMode
language
ttsSpeed
ttsVoice
focusMode
confusableLetterSettings
```

For now use localStorage or Zustand persistence.

The UI should behave as if this is a real account-level profile.

Later the service can be replaced by:

```
GET /api/profile
PUT /api/profile
```

10. PROFILE PAGE (/profile)

Create a complete profile/settings interface.

Sections:

Reading Preferences

- Font
- Font size
- Boldness
- Letter spacing
- Word spacing
- Line spacing
- Paragraph spacing
- Text width
- Alignment

Reading Colors

- Background
- Text
- Highlight

Highlighting

- None
- Current Word
- Current Phrase
- Current Line
- Selective Highlighting
- Confusable Letters

Language

- Preferred reading language
- Preferred audio language

Audio

- Voice
- Speed

Calibration

- View current calibration
- Redo calibration

Buttons:

- Save
 - Reset to calibrated settings
 - Reset to defaults
-

11. DOCUMENT LIBRARY (/library)

Create a document library.

For the prototype, use mock documents.

Each document should contain:

```
id
name
language
date
progress
thumbnail/preview
```

Example:

```
Class 8 Science
80% complete

History Chapter 4
45% complete

English Reading Practice
100% complete
```

Actions:

- Continue Reading
- Open
- Delete

- Rename

The library should later map naturally to backend storage.

12. DOCUMENT UPLOAD

Create a drag-and-drop upload experience.

Accept:

- PDF
- JPG
- JPEG
- PNG

UI:

Drop your document here

or

Choose File

Show:

- file name
- file size
- upload progress
- processing state
- success
- error
- retry

For now, uploaded documents can be processed using a mock OCR service.

Do not call Sarvam directly from the frontend.

13. OCR / DOCUMENT DIGITISATION — MOCK SERVICE

The final application will use Sarvam Document Intelligence.

For now implement:

```
documentService.digitise(file)
```

with a mock implementation.

Mock flow:

```
Upload PDF/Image
  ↓
Processing
  ↓
Mock OCR
  ↓
Extracted structured text
  ↓
Reader
```

The mock response should resemble what the eventual backend will return.

Example:

```
{
  documentId: "doc_001",
  title: "Class 8 Science",
  language: "en-IN",
  content: "...",
  pages: [...],
  status: "completed"
}
```

The frontend must not care whether the response comes from mock OCR or real OCR.

Future API:

```
POST /api/document/digitise
```

14. PDF ORIGINAL VIEW

Support an original document view using PDF.js/react-pdf where appropriate.

The user must be able to switch between:

Original View

and:

Personalized View

Original View:

Display the document approximately as it originally appears.

Personalized View:

Display the extracted content using reading preferences.

The original uploaded document must never be permanently modified.

15. PERSONALIZED READING VIEW

This is the core screen of the application.

Create:

/reader/[documentId]

Layout:

Desktop

Left: Document/navigation panel

Center: Reading content

Right: Personalization controls

Mobile

Single-column reading experience.

Controls should move into drawers/bottom sheets.

The reading content must remain the dominant element.

16. READING CONTROLS

Provide a collapsible control panel.

Controls:

Typography

- Font
- Font size
- Boldness
- Letter spacing
- Word spacing
- Line spacing
- Paragraph spacing
- Text width
- Alignment

Colors

- Background
- Text
- Highlight
- Reading theme

Highlighting

- Current word
- Current phrase
- Current line
- Selective highlighting
- Confusable letters

All changes must update the reading view in real time.

No API call should be required for these changes.

17. ORIGINAL VS PERSONALIZED VIEW

Provide a highly visible toggle:

[Original] [Personalized]

Original:

Show original document/PDF appearance.

Personalized:

Show reflowed text with user preferences.

The two modes must use the same underlying document content.

Switching between them must not reset preferences.

18. FOCUS / READING MODE

Implement a distraction-reduced reading mode.

When enabled:

- hide unnecessary navigation
- enlarge/center reading area
- reduce visual clutter
- keep audio controls available
- maintain the selected reading theme
- maintain highlighting

Support:

Normal Mode
Focus Mode

Allow the user to exit focus mode easily.

19. SELECTIVE HIGHLIGHTING

Allow users to choose what gets highlighted.

Modes:

```
None
Current Word
Current Phrase
Current Line
Selective Highlighting
```

Selective highlighting should allow the user to visually emphasize selected text.

Do not rely only on color to communicate state.

20. CONFUSABLE LETTER HIGHLIGHTING

Implement a frontend feature for commonly confused letter pairs:

```
b / d
p / q
m / w
```

Provide:

```
Confusable Letters
[ON/OFF]

Pairs:
b / d
p / q
m / w
```

When enabled, visually emphasize these characters in the reading content.

The first implementation should be entirely frontend.

Do not claim that the feature diagnoses dyslexia.

Future backend support may store which letter pairs a user frequently struggles with.

21. TEXT-TO-SPEECH

For the frontend prototype use:

```
Web Speech API  
window.speechSynthesis
```

Do not integrate cloud TTS yet.

Create a clean abstraction:

```
ttsService.speak()  
ttsService.pause()  
ttsService.resume()  
ttsService.stop()  
ttsService.setRate()
```

Later this service can be replaced by a backend endpoint using Sarvam Bulbul.

22. AUDIO PLAYER

Create a floating audio dock.

Controls:

- Play
- Pause
- Resume
- Stop
- Previous sentence
- Next sentence
- Replay
- Speed

Speed options:

```
0.5x  
0.75x  
1x  
1.25x  
1.5x  
2x
```

Show current reading state.

Example:

Reading aloud
Sentence 4 of 12

[Previous] [Pause] [Next]

Speed: 1x

23. SYNCHRONIZED READ-ALONG

Use Web Speech API `onboundary` events where supported.

The goal is:

```
Audio
↓
Current spoken word
↓
Highlight matching word in DOM
```

Implement robust text-to-DOM word mapping.

The implementation must account for:

- whitespace differences
- punctuation
- line breaks
- repeated words
- multiple text nodes
- normalized text
- PDF extraction artifacts

Do not assume a simple string index will always map correctly to DOM nodes.

Create a reusable mapping system:

```
source text
↓
tokenize
↓
DOM word spans
↓
Speech boundary index
```

↓
active word index
↓
highlight

If the browser does not provide reliable boundary events, gracefully fall back to sentence-level highlighting or estimated timing.

Do not make the entire reader fail because word-level synchronization is unavailable.

24. AUTO-SCROLL DURING READ-ALONG

When the active word/line moves outside the visible reading area:

- automatically scroll to the active section
- use smooth scrolling where appropriate
- do not aggressively scroll on every word
- maintain user control

If the user manually scrolls, do not fight the user unnecessarily.

25. MULTILINGUAL SUPPORT

Support multilingual reading at the UI level now.

Provide language selection.

Example:

English
Hindi
Bengali
Gujarati
Kannada
Malayalam
Marathi
Odia
Punjabi
Tamil
Telugu

For the prototype, use mock translations.

Create:

```
translationService.translate()
```

Future implementation:

```
POST /api/translate
```

Important:

Translation must NOT reset the user's reading preferences.

Example:

```
English
↓
Hindi

Same:
font
font size
spacing
colors
highlighting
reading mode
```

The reading profile belongs to the user, not to one document language.

26. TRANSLATION UI

Inside the reader provide:

Reading Language

English ▼

When changed:

Translating...

Then display translated content.

Show a clear language indicator.

Allow switching back to the original language.

Preserve the original text.

27. AI TEXT SIMPLIFICATION

This is a **future/probable feature**, but design the frontend now.

Create a button:

Simplify Text

Use mock data for now.

Display:

Original

Complex educational text...

Simplified

Simpler version...

Allow:

[Original] [Simplified]

Never replace the original text permanently.

The frontend should eventually call:

POST /api/simplify

Do not make simplification mandatory for reading.

Do not claim that AI simplification is medically necessary.

28. OPTIONAL ASSESSMENT DOCUMENT

Create an optional assessment upload interface.

The user can choose:

Skip for now

or:

Upload assessment

For now use mock processing.

Future backend:

POST /api/profile/assessment

The system should never diagnose the user.

The uploaded information can only be used as a potential input to personalization.

29. PERSONALIZED SETTINGS MEMORY

For the prototype:

Use localStorage/Zustand persistence.

When the user returns:

```
Load saved reading profile
↓
Apply settings automatically
```

Example:

```
Font: Atkinson Hyperlegible
Font size: user-selected
Spacing: user-selected
Background: Warm Cream
Highlight: Current Word
```

The future backend will store these settings in a database.

30. PER-DOCUMENT SETTINGS

Support two levels:

Global Settings

Apply to every document.

This Document Only

Override settings for one document.

Example:

```
[Save as Global Settings]
[Save for This PDF Only]
```

Use localStorage for the prototype.

Structure the state so these can later map to backend profile/document settings.

31. READING ANALYSIS — FUTURE FEATURE

Do not build sophisticated analytics yet.

However, create the architecture for future reading sessions.

Potential future metrics:

- reading time
- words read
- reading speed
- difficult words
- revisited sections
- pauses
- confusable letter interactions
- preferred settings
- performance with different settings

Future API:

```
POST /api/reading/session
GET /api/reading/analysis
```

For the frontend prototype, optionally show a basic mock reading summary.

Do not claim that this data diagnoses dyslexia.

32. ADAPTIVE PERSONALIZATION — FUTURE FEATURE

Design the UI so future personalization can evolve:

```
Read
↓
Analyze
↓
Identify possible difficulty
↓
Adjust support
↓
Read again
```

Do not automatically make medical or diagnostic decisions.

For the MVP, personalization should be based primarily on explicit user preferences and calibration.

33. OFFLINE / LOW-END DEVICE SUPPORT

This project should remain usable for basic reading functions even with poor connectivity.

Frontend functionality should continue to work when possible:

- saved reading settings
- saved calibration
- text formatting
- focus mode
- reading controls
- basic browser TTS
- previously cached content

Use:

- localStorage
- browser caching where appropriate
- lightweight client-side processing

Do not make the basic personalized reader unnecessarily dependent on the backend.

Future OCR/translation/cloud AI can require connectivity.

Show a graceful offline state rather than a generic error.

34. STATE MANAGEMENT

Use React Context or Zustand.

Create separate logical state domains:

```
userState
readingPreferences
documentState
calibrationState
ttsState
translationState
uiState
```

Avoid one giant global state object.

Reading preferences should be reactive so changes immediately affect the reader.

35. SERVICE ABSTRACTION

This is one of the most important requirements.

Create service interfaces such as:

```
documentService
translationService
ttsService
profileService
calibrationService
simplificationService
readingService
```

Example:

```
documentService.digitise(file)
translationService.translate(text, source, target)
ttsService.speak(text, options)
profileService.getProfile()
profileService.saveProfile(profile)
calibrationService.saveResult(result)
simplificationService.simplify(text)
```

Initially these should use mock implementations.

Do not put mock logic directly inside React components.

Later they can be replaced with:

```
Frontend
↓
FastAPI
↓
Sarvam / Database
```

without changing the component layer.

36. MOCK API BEHAVIOR

Make mock services feel realistic.

For example, OCR should simulate:

```
Uploading
  ↓
Processing document...
  ↓
Extracting text...
  ↓
Formatting content...
  ↓
Complete
```

Translation:

```
Translating...
```

TTS:

```
Preparing audio...
```

Simplification:

```
Simplifying text...
```

Use realistic delays only where they improve the prototype.

Do not make the interface artificially slow.

37. ERROR STATES

Every asynchronous feature needs:

- loading
- success
- empty

- error
- retry

Examples:

OCR failed
[Try Again]

Translation unavailable
[Try Again]

Speech unavailable
[Use browser voice]

Do not allow a failure in one feature to break the entire reading experience.

38. RESPONSIVE DESIGN

Support:

Desktop

Three-column reader where appropriate.

Tablet

Two-column or collapsible controls.

Mobile

Single-column reading view.

Controls should become:

- drawers
- bottom sheets
- compact toolbars

The reading content should remain the primary focus.

39. ACCESSIBILITY

The website itself must be accessible.

Implement:

- semantic HTML
- keyboard navigation
- visible focus states
- accessible labels
- ARIA where appropriate
- sufficient target sizes
- screen-reader-friendly controls
- no color-only state communication
- reduced motion support
- responsive typography
- readable error messages

Do not use animation purely for decoration.

40. PERFORMANCE

Optimize for low-end devices.

Avoid unnecessary:

- large dependencies
- excessive animations
- continuously running effects
- expensive DOM operations

For the reading view, avoid re-rendering the entire document every time the active word changes.

Use efficient word highlighting.

41. COMPONENT STRUCTURE

Use reusable components approximately like:

```
components /  
|
```

```
|— layout/
|   |— Header
|   |— Sidebar
|   └─ MobileNavigation
|
|— reader/
|   |— Reader
|   |— ReadingContent
|   |— ReadingControls
|   |— FocusMode
|   |— HighlightLayer
|   |— AudioDock
|   |— LanguageSelector
|   └─ OriginalPersonalizedToggle
|
|— calibration/
|   |— Calibration
|   |— CalibrationCard
|   |— CalibrationRound
|   └─ CalibrationSummary
|
|— profile/
|   |— Profile
|   |— TypographySettings
|   |— ColorSettings
|   |— HighlightSettings
|   └─ AudioSettings
|
|— documents/
|   |— DocumentUploader
|   |— DocumentCard
|   └─ DocumentLibrary
|
└─ common/
    |— Button
    |— Slider
    |— Modal
    |— Drawer
    |— Dropdown
    └─ LoadingState
```

Adjust the structure if a better architecture is appropriate, but maintain clear separation of concerns.

42. DATA TYPES

Create TypeScript types for:

```
ReadingProfile
ReadingPreferences
Document
DocumentPage
CalibrationResult
CalibrationRound
TranslationResult
TTSSState
ReadingSession
HighlightSegment
User
```

The types should be designed so they can later match backend API schemas.

43. MOCK DATA

Create realistic educational sample content.

Use examples from:

- science
- history
- English reading
- mathematics

Do not use empty lorem ipsum.

The reader should look like a real educational reading application.

44. IMPLEMENTATION PRIORITY

Implement in this order:

Phase 1

Design system + layout

Phase 2

Core Reader

Phase 3

Personalization controls

Phase 4

Color themes

Phase 5

Calibration

Phase 6

Original vs Personalized

Phase 7

Focus Mode + highlighting

Phase 8

Browser TTS + read-along

Phase 9

Document upload + mock OCR

Phase 10

Multilingual UI + mock translation

Phase 11

Profile + settings persistence

Phase 12

Document library

Phase 13

Mock simplification

Phase 14

Optional assessment profile

Phase 15

Offline/cache behavior

Phase 16

Future-feature placeholders

45. DO NOT IMPLEMENT YET

Do not implement:

- real Sarvam API
- real FastAPI backend
- real database
- real cloud OCR
- real cloud translation
- real cloud TTS
- real authentication
- MCP
- production AI simplification
- production reading analytics

Create the interfaces required for these features, but keep the current implementation frontend/mock based.

46. FUTURE BACKEND CONTRACT

Design the frontend service layer so it can eventually connect to endpoints approximately like:

```
GET  /api/profile
PUT  /api/profile
```

```
POST /api/document/digitise
POST /api/document/extract

POST /api/translate

POST /api/tts

POST /api/simplify

POST /api/profile/assessment

GET /api/calibration/samples
POST /api/calibration/result

POST /api/reading/session
GET /api/reading/analysis
```

These are future contracts.

Do not implement the backend now.

47. IMPORTANT PRODUCT PRINCIPLES

The product should communicate:

"Your reading environment should adapt to you."

Do not communicate:

- one font is universally best
- one color is universally best
- the system diagnoses dyslexia
- the system replaces professional assessment

The application is a **personalized reading-support tool**.

48. FINAL ACCEPTANCE CRITERIA

The frontend prototype should allow a user to:

1. Open the application.
2. Log in through a mock login.
3. Complete optional reading calibration.

4. Receive a personalized reading configuration.
5. Upload/select a sample PDF.
6. See realistic mock OCR processing.
7. Read extracted content.
8. Switch between Original and Personalized views.
9. Change font.
10. Change font size.
11. Change spacing.
12. Change text width.
13. Change text/background colors.
14. Change highlighting.
15. Enable confusable-letter highlighting.
16. Enable Focus Mode.
17. Play text using browser TTS.
18. See synchronized word/sentence highlighting where browser support allows.
19. Change reading speed.
20. Change language using mock translation.
21. Preserve personalization when language changes.
22. Save global preferences.
23. Save document-specific preferences.
24. Open the profile and see saved preferences.
25. Open a document library.
26. See mock AI simplification.
27. Work reasonably with previously saved preferences when offline.
28. Work responsively on desktop, tablet and mobile.

The resulting frontend should feel like a **complete working MVP**, even though the AI/backend integrations are mocked.

Most importantly:

Build the frontend around the user experience and service abstraction, not around the current mock implementation.